

Performance Evaluation of Meta-Heuristic Algorithms for Multi-Robot Task Allocation

Mohammed Hany¹, Karim Hassan¹, Jessica Magdy^{1,2}, and Omar M. Shehata^{1,2}

¹Mechatronics Engineering Department, Faculty of Engineering and Materials Science, German University in Cairo, Egypt

²Multi-Robot Systems (MRS) Research Group Cairo, Egypt

{mohammed.hany, karim.ali}@student.guc.edu.eg, {jessica.gergis, omar.mohamad}@guc.edu.eg

Multi-Robot Task Allocation (MRTA) is a Nondeterministic Polynomial-time (NP)-hard optimization problem that requires assigning tasks to multiple robots while minimizing travel cost, maximizing task coverage, and maintaining workload balance. This paper evaluates four meta-heuristic algorithms—Simulated Annealing (SA), Genetic Algorithm (GA), Grey Wolf Optimizer (GWO), and the Bee Algorithm (BA)—for solving MRTA using a unified multi-objective formulation based on distance minimization, task maximization, and Jain's Fairness Index. Each method is tested under multiple parameter settings to analyze convergence behavior, solution quality, and computational efficiency. Simulation results show that the Genetic Algorithm achieves the best overall performance, providing the highest task coverage and fastest convergence while preserving balanced task distribution among robots. Although BA and GWO offer stable and fair solutions and SA produces energy-efficient routes, GA consistently outperforms the other methods in both solution quality and runtime. These results highlight GA as the most suitable choice for scalable and efficient MRTA applications.

Index Terms—Multi-Robot Task Allocation, Simulated Annealing, Genetic Algorithm, Grey Wolf Algorithm, Bee Algorithm.

I. INTRODUCTION

In recent years, the deployment of multi-robot systems has expanded significantly across industrial, agricultural, and defense applications. These systems require intelligent task allocation strategies capable of minimizing operational costs while maximizing efficiency, coverage, and fairness [1]. Traditional deterministic optimization approaches fail to scale due to the combinatorial nature of Multi-Agent Task Allocation, prompting the adoption of meta-heuristic algorithms [2]. This paper contributes a unified comparative analysis that examines not only solution quality but also algorithmic behavior under varying constraints, scalability conditions, and parameter regimes. This work investigates four optimization algorithms implemented for MRTA, analyzing how their design choices temperature schedules, selection strategies, population sizes, and search operators impact solution quality. This paper is divided into five sections: Section II will review previous literature, section III is covering the problem definition and the used algorithms in the paper, Section IV is going to compare the used algorithms and Section V will cover the conclusion and future work recommendations.

II. LITERATURE REVIEW

Recent studies have investigated various optimization-based approaches to address the multi-robot task allocation (MRTA) problem. In [3], Alitappeh and Jeddissaravi proposed a hybrid framework that decomposes the problem into deployment and routing subproblems using a genetic algorithm-based multi-objective optimizer combined with Q-learning. Safe navigation is ensured through the use of Generalized Voronoi Diagrams. Although the proposed method improves routing reliability, it incurs a high computational cost, which may limit its applicability to large-scale systems.

A two-stage MRTA algorithm was presented in [4], where Ant Colony Optimization (ACO) is employed for global task allocation, followed by real-time task negotiation using the Contract Net Protocol (CNP). This approach demonstrates robustness in dynamic environments by enabling adaptive task reassignment. However, its performance is highly dependent on reliable inter-robot communication, which may pose challenges in communication-constrained scenarios.

In [5], Aziz et al. provided a theoretical analysis of MRTA by formalizing problem constraints and deriving approximation bounds for different task allocation variants. The study offers strong mathematical foundations and insights into the computational complexity of MRTA. Nevertheless, the lack of experimental or simulation-based validation limits the practical assessment of the proposed formulations.

A hybrid clustering and optimization approach was introduced in [6], combining the K-Means++ algorithm with Particle Swarm Optimization (PSO). The clustering phase improves workload distribution among robots, while PSO is used to optimize routing and task execution time. Despite achieving improved balance and efficiency, the method is primarily applicable to static environments and does not explicitly address dynamic task reallocation.

A comparative evaluation of Genetic Algorithms (GA) and Ant Colony Optimization (ACO) for large-scale MRTA problems was conducted in [7]. The results indicate that both approaches are capable of solving complex task allocation instances. However, GA exhibits reduced scalability as problem size increases, while ACO requires pre-clustering

to maintain computational efficiency, highlighting inherent trade-offs between solution quality and scalability

III. METHODOLOGY

A. Problem Definition

1) Objective

The primary objective is to allocate N tasks among M robots so as to minimize the total travel distance, maximize the number of completed tasks, and ensure a balanced workload distribution across all robots, while satisfying all operational constraints.

2) Decision Variables

The decision variables are represented as the number of tasks allocated to each robot and the sequence of tasks assigned to each robot.

3) Solution Representation

The solution is represented as a 2D integer array, where each row denotes the sequence of tasks assigned to a robot. Tasks are indexed numerically (e.g., task 1 as 1, task 2 as 2, etc.), while the depot is represented by 0. Accordingly, each robot's route begins and ends with 0. For example a solution represented by the following 2D array:

$$\begin{bmatrix} 0 & 1 & 6 & 3 & 0 \\ 0 & 4 & 2 & 5 & 0 \end{bmatrix} \quad (1)$$

signifies that robot 1 moves in the following order: Depot $\rightarrow$ Task 1 $\rightarrow$ Task 6 $\rightarrow$ Task 3 $\rightarrow$ Depot, while robot 2 moves in the following order: Depot $\rightarrow$ Task 4 $\rightarrow$ Task 2 $\rightarrow$ Task 5 $\rightarrow$ Depot.

4) Objective Function

$$F_1 = \sum_{i=1}^R \sum_{j=0}^N \sum_{k=0}^N x_{ijk} d_{jk} \quad (2)$$

$$F_2 = \frac{(\sum_{i=1}^R \sum_{j=0}^N \sum_{k=0}^N x_{ijk} d_{jk})^2}{R \sum_{i=1}^R (\sum_{j=0}^N \sum_{k=0}^N x_{ijk} d_{jk})^2} \quad (3)$$

$$F_3 = \sum_{i=1}^R \sum_{j=1}^N y_{ij} \quad (4)$$

$$\text{Total Cost} = \frac{W_1}{F_1} + W_2 \times F_2 + W_3 \times F_3 \quad (5)$$

where x_{ijk} is 1 when robot (i) takes task (k) after finishing task (j), otherwise 0, d_{jk} is the distance between task (j) and task (k) and y_{ij} is 1 when robot (i) takes task (j), otherwise 0, W_1 , W_2 and W_3 represent the weights for each objective term which were chosen to be 15000, 100 and 1 respectively. Equation (2) calculates the total travel distance covered by robots and is minimized in the optimization process. Equation (3) measures the workload balance among the robots using Jain's Fairness Index which should be maximized. Equation (4) finds the total number of tasks taken by the robots and is formulated as a maximization objective. Equation (5) calculates the total cost using the values calculated from the previous three equations which is to be maximized.

The weights W_1, W_2 and W_3 were selected to normalize the relative magnitudes of the three objective components. Preliminary experiments showed that the travel distance term F_1 dominated numerically, requiring a larger scaling factor to maintain balanced contribution in the aggregated objective. The selected values provided stable convergence and meaningful trade-offs between coverage, distance, and fairness. Distance is computed as Manhattan distance and the aim is to maximize the objective function. [8]

5) Constraints

The optimization is subject to a set of constraints modeled using the following equations:

$$\sum_{j=0}^N y_{ij} w_j \leq W_{max}, i = [1, 2, 3, \dots, R] \quad (6)$$

$$\sum_{i=1}^R \sum_{j=1}^N x_{ijk} = 1, k = [1, 2, 3, \dots, N] \quad (7)$$

$$\sum_{i=1}^R \sum_{k=1}^N x_{ijk} = 1, j = [1, 2, 3, \dots, N] \quad (8)$$

$$\sum_{j=0}^N \sum_{k=0}^N x_{ijk} d_{jk} \leq D_{max}, i = [1, 2, 3, \dots, R] \quad (9)$$

Where w_j is the weight of the part carried by the robot after finishing task (j). Equation (6) checks that no robot carries more than its maximum carrying capacity load. Equations (7) and (8) check that each task except for the depot should be performed once. Equation (9) checks that no robot covers more than a certain distance. [8]

B. Simulated Annealing

Simulated Annealing operates by probabilistically accepting worse solutions early in the search, allowing the algorithm to escape local minima. The cooling schedule plays a critical role in balancing exploration and exploitation [9]. The algorithm has been extensively adopted in comparative analyses, such as in [10]. In this study, multiple schedules (linear and geometric) were tested under varying initial and final temperatures.

SA's performance is heavily influenced by:

- Neighborhood operators, which determine how solution permutations evolve.
- Constraint repair, ensuring feasibility without discarding potentially valuable candidate solutions.
- Cooling schedule selection, impacting convergence stability.

C. Genetic Algorithm

The Genetic Algorithm is a popular population-based meta-heuristic algorithm that evolves a population of task-allocation chromosomes through selection, crossover, and mutation [11]. Due to its robustness and broad adoption, GA is commonly included in comparative assessments of metaheuristic approaches [12]. The specific OX1 (Order Crossover) method used in this study is highly effective for permutation-based

optimization because it preserves relative ordering while introducing meaningful diversity.

The GA's performance is strongly affected by:

- Population size, which dictates genetic diversity.
- Elite percentage, affecting exploitation strength.
- Mutation strategy, responsible for escaping sub-optimal local structures.

D. Grey Wolf Algorithm

Grey Wolf Optimization models leadership hierarchy and cooperative hunting patterns. The algorithm's unique ability to adapt search aggressiveness through its linearly decaying parameter ' α ' allows for controlled convergence [13].

However, applying GWO to discrete MRTA requires careful discretization. This study utilizes rounding and clamping to map continuous values into task indices. Despite the simplicity of this transformation, results indicate that discretization quality significantly affects overall performance. [14]

E. Bee Algorithm

The Bee Algorithm employs a dual exploration-exploitation process using scout bees (global search) and forager bees (local search). Elite sites attract more bees, enabling deeper exploitation of promising solutions. [15] [16]

The BA's performance is shaped by:

- Number of elite and selected sites, controlling intensification strength.
- Search radius, affecting local neighborhood perturbation.
- Scout proportion, responsible for exploring new unvisited regions.

IV. RESULTS AND DISCUSSION

For each algorithm three cases are going to be studied by varying some of the algorithm's parameters. After that the four algorithms are going to be compared on the basis of average cost, standard deviation, average runtime and best cost.

A. Simulated Annealing

Table I shows the used parameters in each case. For case 1 as shown in figure 1, the algorithm showed smooth, monotonic convergence as the temperature decreased, producing stable cost improvements without major oscillations. Task allocation remained balanced among the 10 robots, with most tasks completed and minimal overlap. The temperature schedule demonstrated gradual cooling, confirming proper control of exploration and exploitation. For case 2, switching to a geometric schedule accelerated early convergence but caused occasional cost fluctuations near the end of the run. The rapid temperature decay reduced exploration too soon, leading

Case	Initial Temperature	Final Temperature	Cooling Schedule
1	1000	100	Linear
2	1000	100	Geometric
3	1000	0.1	Linear

TABLE I: SA Case Parameters

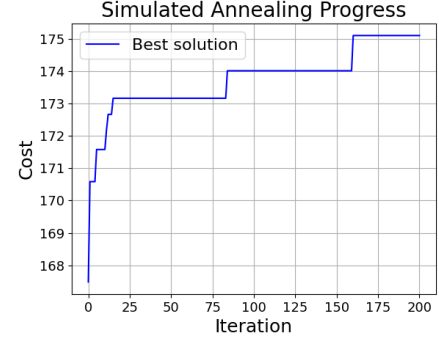


Fig. 1: SA current and best solutions' cost graph for case 1

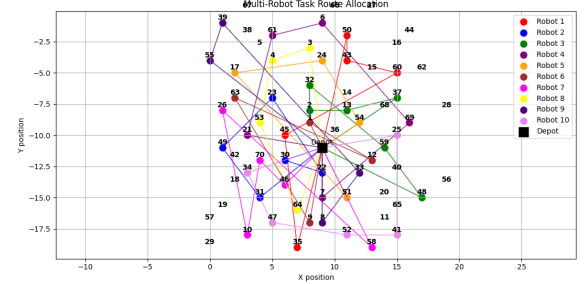


Fig. 2: SA best solution robot allocation for case 1

to premature convergence in several trials. Nevertheless, this case reached competitive costs faster than Case 1, proving geometric cooling's advantage in time-critical scenarios where speed outweighs global-search depth. For case 3, lowering the final temperature forced the system into a highly exploitative phase. SA explored less but refined existing allocations thoroughly, achieving one of the most energy-efficient task routes. However, diminishing acceptance probability led to stagnation after mid-iterations, with limited new task redistribution.

B. Genetic Algorithm

Table II shows the used parameters in each case. Figure 3 shows the convergence curve for the three cases. For case 1, high crossover with modest mutation produced fast, steady convergence. Elitism preserved the best sequences each generation, so the GA progress curve climbed smoothly without dramatic dips. Task-allocation matrices showed even work distribution across robots and short inter-task hops thanks to OX1 preserving relative order. This case is a reliable reference for stability and speed. For case 2, reducing crossover and raising mutation to 50% increased diversity significantly. Early generations explored more aggressively (progress curve had small fluctuations), then settled to strong solutions after repair. This setting found alternative route patterns that the baseline

Case	Elite%	Crossover%	Mutant%
1	10	80	10
2	10	40	50
3	60	20	20

TABLE II: GA Case Parameters

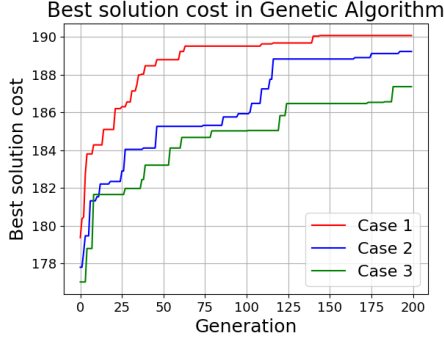


Fig. 3: GA best solution cost graph for cases 1, 2 and 3

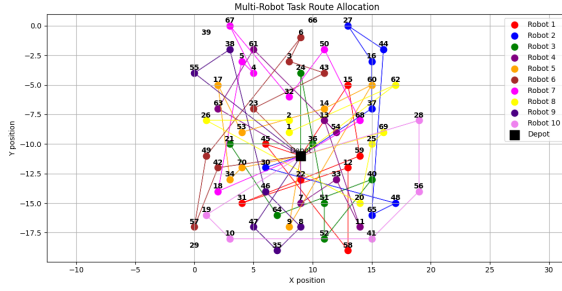


Fig. 4: GA best solution robot allocation for case 1

missed, at the cost of slightly noisier convergence. It is useful when the search space is deceptive or multi-modal. For case 3, raising elitism to 60% amplified exploitation. The best genetic material propagated quickly, but diversity dropped. When initial elites are strong this setting excels; when they're mediocre, it risks converging early. Still, for our 70-task instances it consistently produced high-quality, fair allocations.

C. Grey Wolf Algorithm

Table III shows the used parameters in each case. For case 1 as shown in figure 5, the Grey Wolf algorithm demonstrated

Case	Population	Maximum Iterations
1	100	200
2	200	200
3	100	400

TABLE III: GWO Case Parameters

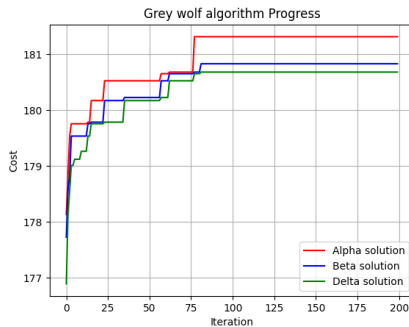


Fig. 5: GWO best solution cost graph for case 1

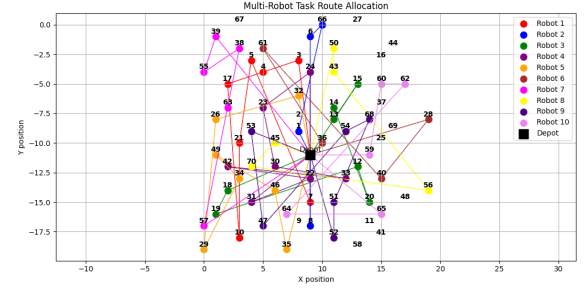


Fig. 6: GWO best solution robot allocation for case 1

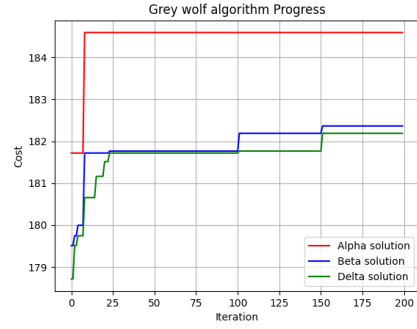


Fig. 7: GWO best solution cost graph for case 2

steady and stable convergence, resulting in feasible task assignments with good fairness among agents. However, the limited exploration capability lead to early pack convergence, which caused partial stagnation and restricted the algorithm's ability to further improve solution quality. For case 2 as shown in figure 7, the population of wolves was doubled, leading to a notable increase in solution diversity. The larger population enhanced exploration during the early iterations and improved exploitation in the later stages, resulting in higher task coverage compared to Case 1 and a reduced tendency to become trapped in local minima due to the broader search of the solution space. Although the increased population caused a rise in computation time, the overall runtime remained within acceptable limits. For case 3 as shown in figure 8, doubling the iteration count did not yield proportional performance improvements. Only minimal gains

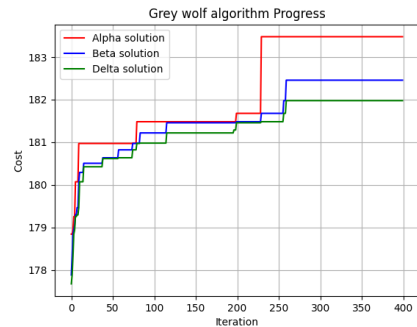


Fig. 8: GWO best solution cost graph for case 3

Case	Population	Elite%	Neighboring%	Remaining%
1	100	0.1	0.5	0.4
2	100	0.5	0.4	0.1
3	200	0.1	0.5	0.4

TABLE IV: BA Case Parameters

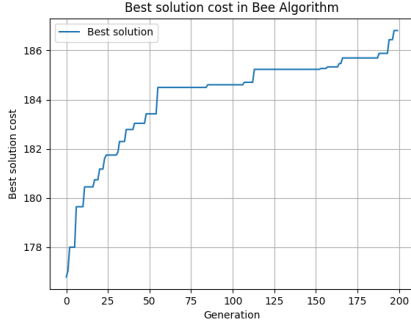


Fig. 9: BA best solution cost graph for case 1

were observed beyond approximately 200 iterations, indicating diminishing returns, while the computational cost increased significantly. Consequently, the final solution quality and cost remained largely comparable to those of Case 1, suggesting that extending the iteration limit alone is inefficient for further optimization.

D. Bee Algorithm

Table IV shows the used parameters in each case. For case 1 as shown in figure 9, the convergence behavior exhibits a sharp improvement from 176 to 182 within the first 20 generations, followed by smooth and steady gains up to approximately 184 by generation 50. After this point, the algorithm enters a long plateau around a cost of 185, indicating saturation of the global search. Minor improvements are observed beyond 180–200 generations, with the final cost reaching approximately 187. Overall, this case demonstrates strong and well-balanced performance, where effective early exploration is complemented by gradual exploitation in later stages. The convergence curve is highly stable, making this case a suitable baseline for comparison with all other scenarios. For case 2 as shown in figure 11, fast early improvement is exhibited, comparable to or slightly faster than Case 1; however, the algorithm experiences increased stagnation in later stages, particularly between

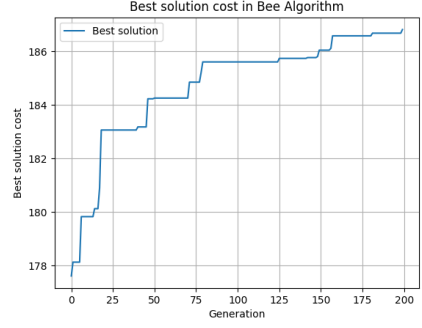


Fig. 11: BA best solution cost graph for case 2

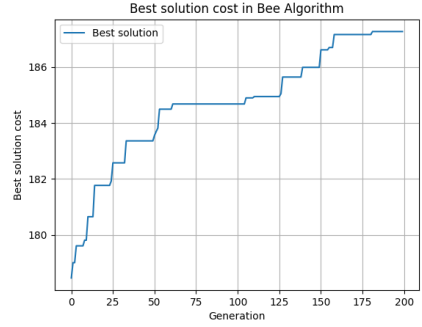


Fig. 12: BA best solution cost graph for case 3

generations 60 and 140. The final solution quality is slightly inferior to that of Case 1, reaching approximately 186.5 instead of 187. The excessive number of elite sites leads to over-exploitation, while the limited number of scout bees reduces exploration capability. As a result, the algorithm converges quickly to good solutions but becomes trapped in local optima, causing premature convergence and a marginally worse overall performance compared to Case 1. For case 3 as shown in figure 12, early convergence behavior similar to Case 1 is found, but with a noticeably smoother progression. By around generation 50, the cost reaches approximately 185, which is slightly better than Case 1 at the same stage. Unlike earlier cases, the algorithm continues to improve more reliably in later generations, ultimately peaking at around 187.5, outperforming both Case 1 and Case 2. The improved performance can be attributed to doubling the population, which enhances exploration coverage, increases solution diversity, and expands the number of sampled neighborhoods, thereby enabling more effective search of the solution space.

E. Comparison of the Four Algorithms and Key Findings

The average values for the four algorithms were calculated on the basis of 10 runs using the base case (case 1) parameters

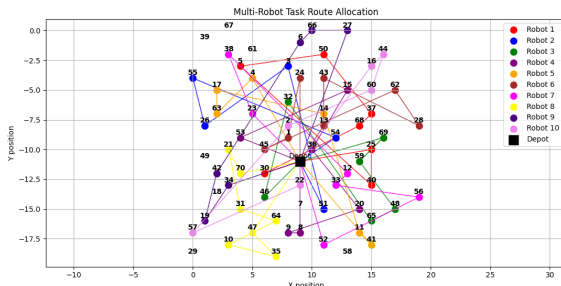


Fig. 10: BA best solution robot allocation for case 1

Algorithm	Cost (mean $\pm$ std)	Time (s)	Best Cost
SA	175.13 $\pm$ 1.08	28.76	176.66
GA	193.59 $\pm$ 0.99	15.62	194.7
GWO	183.77 $\pm$ 0.58	35.07	184.54
BA	186.667 $\pm$ 0.451	116.044	187.37

TABLE V: Performance Comparison Summary

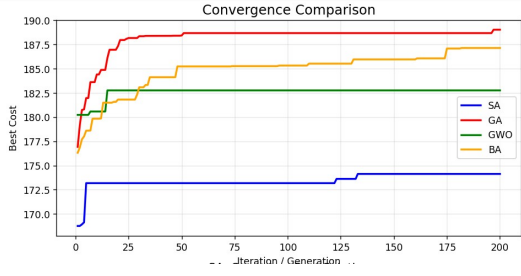


Fig. 13: Convergence Curve of the Four Algorithms

for each algorithm. Table V compares the performance of the four algorithms, while figure 13 shows the best solution's cost progression in each iteration for each algorithm. From the previous simulations, it is shown that the Genetic Algorithm (GA) achieved the highest cost across all runs, with a best value of 194.70, outperforming the Bee Algorithm (187.37), Grey Wolf Optimizer (184.54), and Simulated Annealing (176.66). GA also demonstrated superior task coverage, covering 43.1% more tasks than SA and 8.5% more than GWO, while the Bee Algorithm achieved moderate coverage, exceeding SA but remaining below GA and GWO. In terms of computational efficiency, GA was significantly faster than both SA and GWO, whereas the Bee Algorithm exhibited the longest runtime due to its multi-level neighbourhood search structure. Each algorithm presents distinct solution quality trade-offs: GA excels in maximizing task coverage and speed, GWO offers balanced and repeatable performance, SA provides the most energy-efficient routing with minimal travel distance, and the Bee Algorithm delivers high-quality and fair solutions at the expense of slower convergence. Regarding repeatability, the Bee Algorithm showed the lowest standard deviation in cost, indicating the highest stability, followed by GWO, GA, and SA. All algorithms achieved high fairness values above 0.98, with GA, GWO, and BA demonstrating particularly well-balanced task distributions. As the population size increased (GWO Case 2, BA Case 3), solution quality improved but runtime scaled approximately linearly, indicating a trade-off between exploration depth and computational cost. GA demonstrated the lowest sensitivity to parameter changes, whereas SA showed noticeable performance variation with cooling schedule selection. Algorithms with strong population diversity mechanisms (GA mutation, BA scouts) avoided premature convergence more effectively than trajectory-based SA.

V. CONCLUSION AND FUTURE RECOMMENDATIONS

In conclusion, this paper studied the problem of Multi-Robot Task Allocation (MRTA) using four different algorithms: Simulated Annealing, Genetic Algorithm, Grey Wolf Algorithm and Bee Algorithm.

The Genetic Algorithm demonstrated superior performance for multi-robotics task optimization when the objective was to maximize task coverage while maintaining computational efficiency. However, the choice of algorithm should be guided by specific application requirements:

- For maximum task completion: Genetic Algorithm

- For balanced performance and repeatability: Grey Wolf Optimizer
- For energy conservation: Simulated Annealing
- For high fairness and solution quality (at higher computation time): Bee Algorithm

Future work will investigate hybrid optimization frameworks that leverage the complementary strengths of the studied algorithms, incorporate adaptive parameter tuning mechanisms, and integrate machine learning techniques to enhance MRTA solution quality and robustness. Additionally, subsequent studies could compare the used meta-heuristics to other classical MRTA methods.

REFERENCES

- [1] G. A. Korsah, A. Stentz, and M. B. Dias, "A comprehensive taxonomy for multi-robot task allocation," *The International Journal of Robotics Research*, vol. 32, no. 12, pp. 1495–1512, 2013.
- [2] F. Ahmed, H. Amr, M. Khalil, M. Mohamed, Y. Amir, Z. Yasser, J. Magdy, and O. M. Shehata, "Route to clean cities: Waste collection optimization with uavs," in *2025 15th International Conference on Electrical Engineering (ICEENG)*. IEEE, 2025, pp. 1–6.
- [3] R. J. Alitappeh and K. Jeddisaravi, "Multi-robot exploration in task allocation problem," *Applied Intelligence*, vol. 52, no. 2, pp. 2189–2211, 2022.
- [4] J. Huang, Q. Song, and Z. Xu, "Multi robot cooperative rescue based on two-stage task allocation algorithm," in *Journal of Physics: Conference Series*, vol. 2310, no. 1. IOP Publishing, 2022, p. 012091.
- [5] H. Aziz, "Algorithmic and complexity aspects of simple coalitional games," Ph.D. dissertation, University of Warwick, 2009.
- [6] Y. Yuan, P. Yang, H. Jiang, and T. Shi, "A multi-robot task allocation method based on the synergy of the k-means++ algorithm and the particle swarm algorithm," *Biomimetics*, vol. 9, no. 11, p. 694, 2024.
- [7] M. Shelkamy, C. M. Elias, D. M. Mahfouz, and O. M. Shehata, "Comparative analysis of various optimization techniques for solving multi-robot task allocation problem," in *2020 2nd Novel Intelligent and Leading Emerging Sciences Conference (NILES)*. IEEE, 2020, pp. 538–543.
- [8] S. Wang, Y. Liu, Y. Qiu, Q. Zhang, F. Huo, Y. Huangfu, C. Yang, and J. Zhou, "Cooperative task allocation for multi-robot systems based on multi-objective ant colony system," *IEEE Access*, vol. 10, pp. 56375–56387, 2022.
- [9] D. Bertsimas and J. Tsitsiklis, "Simulated annealing," *Statistical science*, vol. 8, no. 1, pp. 10–15, 1993.
- [10] M. Elsheikh, A. Yonis, J. Magdy, and O. M. Shehata, "Advancing sustainable energy: Analyzing stochastic algorithms and q-learning for wind farm layout optimization," in *2024 5th International Conference on Artificial Intelligence, Robotics and Control (AIRC)*. IEEE, 2024, pp. 128–132.
- [11] T. V. Mathew, "Genetic algorithm," *Report submitted at IIT Bombay*, vol. 53, pp. 18–19, 2012.
- [12] S. Mahdi, A. Yasser, O. Hashem, A. Hatem, J. Magdy, and O. M. Shehata, "Hybrid energy system planning: Optimizing solar and wind power for maximum efficiency and cost reduction," in *2025 15th International Conference on Electrical Engineering (ICEENG)*. IEEE, 2025, pp. 1–6.
- [13] S. Mirjalili, S. M. Mirjalili, and A. Lewis, "Grey wolf optimizer," *Advances in engineering software*, vol. 69, pp. 46–61, 2014.
- [14] X. Luo, Q. Qian, and Y. F. Fu, "Improved genetic algorithm for solving flexible job shop scheduling problem," *Procedia Computer Science*, vol. 166, pp. 480–485, 2020.
- [15] X.-S. Yang, "Engineering optimizations via nature-inspired virtual bee algorithms," in *International Work-Conference on the Interplay Between Natural and Artificial Computation*. Springer, 2005, pp. 317–323.
- [16] D. T. Pham, A. Ghanbarzadeh, E. Koç, S. Otri, S. Rahim, and M. Zaidi, "The bees algorithm—a novel tool for complex optimisation problems," in *Intelligent production machines and systems*. Elsevier, 2006, pp. 454–459.